Tìm kiếm tuyến tính và nhị phân

Tài liệu học tập chi tiết · Lý thuyết, ví dụ, thực hành và hướng giải

Vị trí bài học trong khóa

- Khóa học: Cấu trúc dữ liệu và giải thuật cơ bản.
- Chương 2: Tìm kiếm và sắp xếp.
- Các bài trong chương: Tìm kiếm tuyến tính và nhị phân; Sắp xếp và độ phức tạp.
- Đọc lý thuyết, tự làm ví dụ rồi đối chiếu hướng giải; sau đó mới làm quiz. Bài thực hành tổng hợp cuối khóa dùng để kết nối kiến thức giữa các chương.

> **Mục tiêu bài học**
>
> Xác định điều kiện áp dụng và so sánh chi phí tìm kiếm.

Kiến thức nền

Tìm kiếm tuyến tính kiểm tra lần lượt các phần tử; dùng được với dữ liệu chưa sắp xếp. Trường hợp xấu nhất phải xét n phần tử nên là O(n). Kết quả tìm kiếm nên quy ước rõ vị trí tìm thấy hoặc dấu hiệu không có dữ liệu.

Tìm kiếm nhị phân trên mảng đã sắp xếp so sánh mục tiêu với phần tử giữa rồi bỏ một nửa khoảng tìm kiếm. Số bước so sánh trong trường hợp xấu nhất là O(log n). Điều kiện sắp xếp phải phù hợp phép so sánh; không được áp dụng trực tiếp lên một mảng tùy ý chưa sắp xếp.

Trong biến thể dùng hai biên đóng, nếu mục tiêu nhỏ hơn phần tử giữa thì cập nhật biên phải thành mid-1; nếu lớn hơn thì biên trái thành mid+1. Cần bảo đảm khoảng tìm kiếm giảm sau mỗi bước để tránh vòng lặp vô hạn.

1. Hợp đồng hàm tìm kiếm

Quy định trả chỉ số hay giá trị, và trả gì khi không tìm thấy. Nếu dữ liệu có phần tử trùng, yêu cầu có thể là một vị trí bất kỳ, vị trí đầu hoặc cuối; đó là ba hợp đồng khác nhau. Tìm nhị phân cần mảng đã sắp theo cùng phép so sánh đang dùng.

2. Cài đặt có biên rõ ràng

```
def binary_search(values, target):
    left, right = 0, len(values) - 1
    while left <= right:
        mid = left + (right - left) // 2
        if values[mid] == target:
            return mid
```

```
        if values[mid] < target:
            left = mid + 1
        else:
            right = mid - 1
    return -1

assert binary_search([], 3) == -1
assert binary_search([2, 5, 8, 13, 21], 13) == 3
assert binary_search([2, 5, 8, 13, 21], 7) == -1
```

Bất biến: nếu target tồn tại ở phần chưa loại thì nó nằm trong khoảng đóng left..right. mid±1 bảo đảm khoảng giảm. Không trộn công thức biên đóng này với thuật toán khoảng nửa mở mà không sửa điều kiện tương ứng.

3. Truy vết và phân tích

Trong [2,5,8,13,21], tìm 13: mid=2 có 8 nên left=3; mid=3 có 13 nên trả 3. Khoảng giảm một nửa nên O(log n) lần so sánh trên mảng truy cập O(1). Nếu phải sắp xếp đầu vào cho chỉ một lần tìm, cần tính thêm chi phí sắp xếp.

4. Bài tập mở rộng có đáp án định hướng

Sửa hàm để trả vị trí đầu của 5 trong [1,5,5,5,9]. Khi gặp bằng, lưu ứng viên và tiếp tục tìm bên trái; không trả ngay. Kiểm thử đầu, cuối, rỗng, một phần tử và tất cả bằng nhau. Giải thích vì sao đổi left=mid có thể làm lặp vô hạn khi left sát right.

> **Bài tập vận dụng bổ sung**
>
> tìm 13 trong [2,5,8,13,21,34,55]. Ghi hai biên và phần tử giữa ở từng bước. Thử tiếp một giá trị không có trong mảng.

Câu hỏi tự kiểm tra

1. Tìm kiếm nhị phân trực tiếp trên mảng cần điều kiện nào?
2. Trường hợp xấu nhất của tìm kiếm tuyến tính là gì?
3. Trường hợp xấu nhất của tìm kiếm nhị phân trên mảng là gì?
4. Nếu mục tiêu nhỏ hơn phần tử giữa của mảng tăng dần, nên xét đâu?
5. Vì sao phải cập nhật biên sau mỗi bước tìm kiếm nhị phân?

Gợi ý đáp án và điểm cần nhớ

- Mảng đã sắp xếp theo phép so sánh sử dụng. Thứ tự cho phép loại bỏ một nửa không gian tìm kiếm.
- O(n). Có thể phải đọc hết mảng trước khi kết luận.
- O(log n). Mỗi bước giảm khoảng tìm kiếm xuống khoảng một nửa.
- Nửa bên trái. Mọi phần tử bên phải giữa đều không nhỏ hơn phần tử giữa.

- Để khoảng tìm kiếm giảm và thuật toán kết thúc. Nếu biên không tiến triển, thuật toán có thể lặp vô hạn.

Cách tự đánh giá sau bài

- Giải thích được thuật ngữ mà không đọc lại định nghĩa.
- Thực hiện lại ví dụ với dữ liệu khác và nêu được giới hạn áp dụng.
- Hoàn thành bài tập, đối chiếu bằng chứng hoặc đầu ra với hướng giải.
- Nếu còn nhầm, ghi câu hỏi cụ thể để hỏi giảng viên hoặc AI Tutor cùng ngữ cảnh bài.

Các câu tự kiểm tra có thể trùng trọng tâm quiz. Làm đúng câu đã đọc không tự chứng nhận thành thạo; cần thêm bài vận dụng và minh chứng thực hành.

Tài liệu đọc thêm

- https://opendatastructures.org/
- https://docs.python.org/3/tutorial/datastructures.html
- https://docs.python.org/3/library/heapq.html

Tìm kiếm tuyến tính và nhị phân — Thực hành và hỏi đáp

Phần bổ sung cho bài học · biên soạn 29/10/2026

Tình huống, ví dụ và tự kiểm tra

Tình huống: tìm nhị phân trả sai vì dữ liệu chưa sắp xếp. Binary search cần thứ tự và bất biến rõ ràng; với dữ liệu trùng, phải quy định cần bất kỳ vị trí hay vị trí đầu tiên.

```
from bisect import bisect_left
values = [1, 3, 3, 8]
i = bisect_left(values, 3)
assert i == 1
j = bisect_left(values, 5)
assert j == 3 and values[j] != 5
```

Tự luyện: tìm 0, 1, 3, 9 và trên mảng rỗng. **Hướng giải:** bisect trả vị trí chèn, không tự chứng minh giá trị tồn tại; kiểm tra chỉ số trong miền và phần tử bằng đích trước khi kết luận tìm thấy.

Nguồn học tiếp

https://docs.python.org/3/library/collections.html